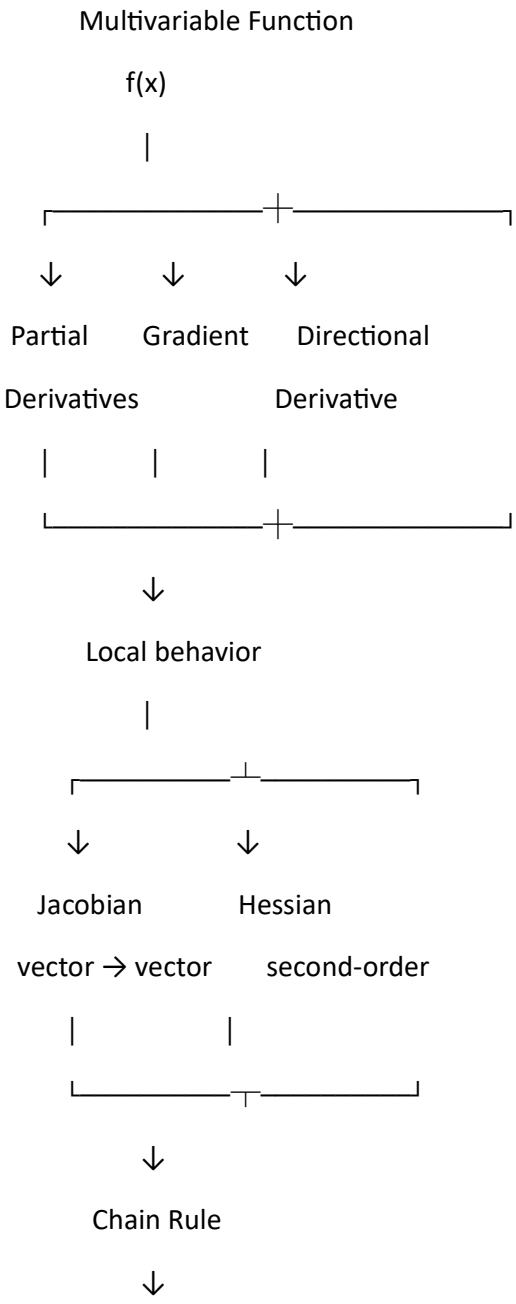
Absolutely. I'll turn the uploaded **"Derivatives and Gradients"** material into an **80/20 interview-oriented set of notes** rather than reproducing the chapter line-by-line.

The source covers partial derivatives, gradients, total derivatives, directional derivatives, Jacobians, Hessians, and the multivariable chain rule, with the chain rule specifically connected to neural-network training/backpropagation.

Derivatives & Gradients — 80/20 Interview Notes

0. The Big Picture

Before memorizing formulas, understand this:



Backpropagation

↓

Gradient Descent

↓

Model Training

Interview mental model:

A derivative tells us how a function changes.
In multiple dimensions, partial derivatives measure change along individual coordinates, the gradient collects those changes into a vector, the Jacobian generalizes this to vector-valued functions, the Hessian captures second-order curvature, and the chain rule connects derivatives through compositions.

The source explicitly describes the gradient as the multivariable analogue used by gradient descent and identifies the multivariable chain rule as essential for practical gradient computation.

---

1. Partial Derivative ⭐ P0

Definition

For

$$f: \mathbb{R}^n \to \mathbb{R}$$

the **partial derivative with respect to** x_i measures how f changes when we change x_i while keeping all other variables fixed.

$$\boxed{\frac{\partial f}{\partial x_i}}$$

Formal definition:

$$\frac{\partial f}{\partial x_i}(x) = \lim_{h \to 0} \frac{f(x + he_i) - f(x)}{h}$$

where e_i changes only the i-th coordinate.

Simple interpretation

Suppose:

$$f(x, y) = x^2 + 3y^2$$

Then:

$$\frac{\partial f}{\partial x} = 2x$$

because $y$is treated as a constant.

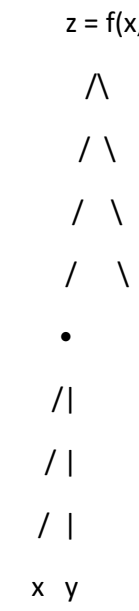
And:

$$\frac{\partial f}{\partial y} = 6y$$

because $x$is treated as a constant.

Diagram

Imagine a surface:



Partial derivative in x:

move → along x-axis

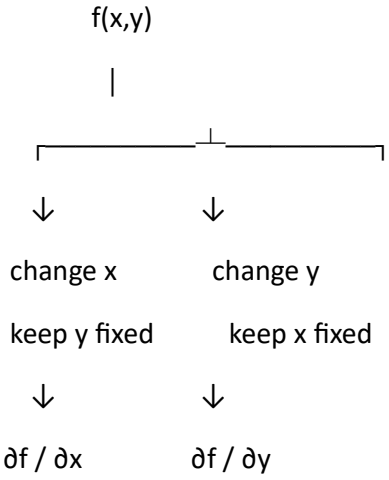
keep y fixed

Partial derivative in y:

move → along y-axis

keep x fixed

Another useful mental model:



ML Example

Suppose a model has two parameters:

$$L(w_1, w_2) = w_1^2 + 2w_2^2$$

Then:

$$\frac{\partial L}{\partial w_1} = 2w_1$$
$$\frac{\partial L}{\partial w_2} = 4w_2$$

At:

$$(w_1, w_2) = (2, 3)$$

we get:

$$\frac{\partial L}{\partial w_1} = 4$$
$$\frac{\partial L}{\partial w_2} = 12$$

So changing w_2 locally affects the loss more strongly than changing w_1.

Interview Question

Q: What is a partial derivative?

Strong answer

A partial derivative measures the rate of change of a multivariable function with respect to one variable while holding the other variables constant. In machine learning, if the loss depends on millions of parameters, each partial derivative tells us how changing one parameter affects the loss locally.

---

2. Gradient ⭐ ⭐ ⭐ P0

This is one of the **most important concepts for an ML interview**.

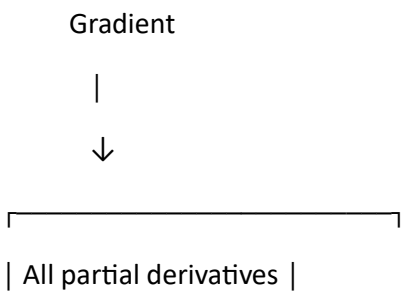
Definition

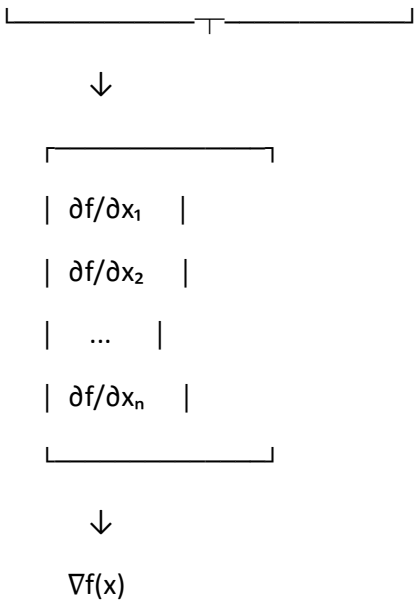
The gradient is the vector containing all partial derivatives:

$$\nabla f(x) = \begin{bmatrix} \frac{\partial f}{\partial x_1} \\ \frac{\partial f}{\partial x_2} \\ \vdots \\ \frac{\partial f}{\partial x_n} \end{bmatrix}$$

The source defines the gradient exactly this way.

Diagram





Think:

Partial derivatives = individual pieces
Gradient = all pieces collected into one vector

Example

$$f(x,y) = x^2 + y^2$$

Partial derivatives:

$$\frac{\partial f}{\partial x} = 2x$$
$$\frac{\partial f}{\partial y} = 2y$$

Therefore:

$$\boxed{\nabla f(x,y) = \begin{bmatrix} 2x \\ 2y \end{bmatrix}}$$

At $(2,3)$:

$$\nabla f(2,3) = \begin{bmatrix} 4 \\ 6 \end{bmatrix}$$

---

3. What Does the Gradient Mean?

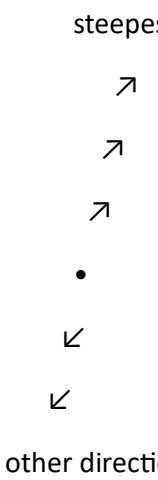
This is more important than memorizing the formula.

The gradient tells us:

1. Direction of maximum increase

The gradient points in the direction where the function increases fastest.

The source derives that the normalized gradient maximizes the directional derivative.



2. Magnitude of gradient

$$|| \nabla f ||$$

represents the maximum rate of increase over unit directions.

3. Negative gradient

$$-\nabla f$$

points toward the direction of **steepest local decrease**.

That is why gradient descent uses:

$$\boxed{w_{new} = w - \eta \nabla L(w)}$$

---

4. Gradient Descent ⭐⭐⭐

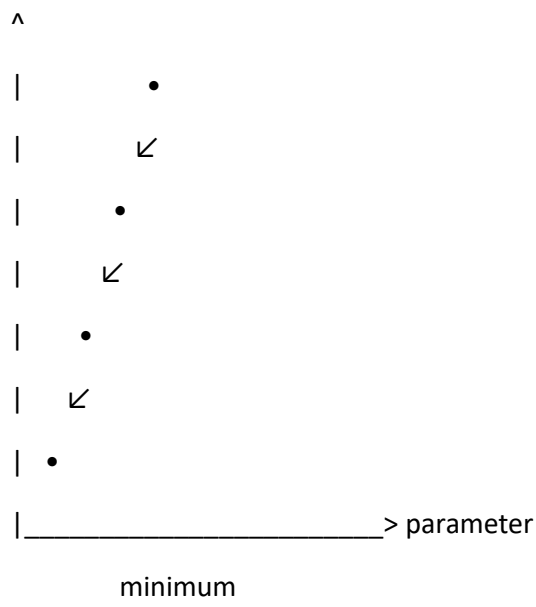
This is where calculus becomes ML engineering.

Suppose:

$$L(w)$$

is your loss.

We want to minimize it.



Update:

$$\boxed{w_{t+1} = w_t - \eta \nabla L(w_t)}$$

where:

- w_t= current parameters
- η= learning rate
- $\nabla L(w_t)$= gradient

Interview answer

The gradient points toward steepest increase, so we subtract it to move toward steepest local decrease. The learning rate controls the step size.

---

5. Directional Derivative ⭐ ⭐ P1

A partial derivative asks:

What happens if I move along one coordinate axis?

A directional derivative asks:

What happens if I move in **any direction**?

Definition:

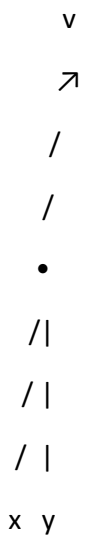
$$\boxed{D_v f(a) = \lim_{h \to 0} \frac{f(a + hv) - f(a)}{h}}$$

The source introduces directional derivatives precisely as derivatives along arbitrary directions.

If f is differentiable:

$$\boxed{D_v f(a) = \nabla f(a)^T v}$$

Diagram



Gradient → tells us how function changes

in the direction of v:

$\nabla f \cdot v$

Example

Let:

$$f(x, y) = x^2 + y^2$$

At:

$$(1, 2)$$

gradient:

$$\nabla f = \begin{bmatrix} 2 \\ 4 \end{bmatrix}$$

Suppose:

$$v = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

Then:

$$D_v f = \nabla f^T v$$
$$= [2, 4] \begin{bmatrix} 1 \\ 0 \end{bmatrix} = 2$$

---

6. Total Derivative ⭐ ⭐ P1

This is often confused with partial derivatives.

Core idea

Partial derivatives tell us individual rates of change.

The **total derivative** describes the **best local linear approximation of the entire function**.

The source emphasizes this distinction: total differentiability is stronger than merely having partial derivatives.

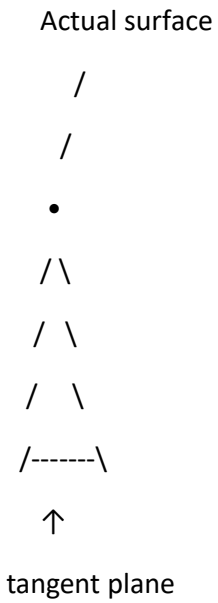
For:

$$f: \mathbb{R}^n \to \mathbb{R}$$

the local approximation is:

$$\boxed{f(x) \approx f(a) + \nabla f(a)^T (x - a)}$$

Diagram



Gradient gives the coefficients

of the tangent-plane approximation.

Interview distinction

Concept	**Meaning**
Partial derivative	Change in one coordinate
Gradient	All partial derivatives collected
Directional derivative	Change in an arbitrary direction
Total derivative	Best local linear approximation

---

7. Partial Derivatives ≠ Differentiability

Important interview trick.

Having all partial derivatives does **not necessarily mean** the function is totally differentiable.

The source explicitly provides an example where all partial derivatives exist at zero but the total derivative does not.

So:

Partial derivatives exist

↓

NOT

↓

Automatically totally differentiable

But:

Totally differentiable

↓

Partial derivatives exist

---

8. Higher-Order Derivatives

Once you have first derivatives, differentiate again.

For example:

$$f(x, y) = x^2 + 3xy + y^2$$

First derivatives:

$$f_x = 2x + 3y$$
$$f_y = 3x + 2y$$

Second derivatives:

$$f_{xx} = 2$$
$$f_{xy} = 3$$
$$f_{yx} = 3$$
$$f_{yy} = 2$$

These second derivatives form the **Hessian**.

---

9. Hessian Matrix ⭐ ⭐ P1

Definition

For scalar function:

$$f: \mathbb{R}^n \to \mathbb{R}$$

the Hessian is:

$$H_f = \begin{bmatrix} \frac{\partial^2 f}{\partial x_1^2} & \frac{\partial^2 f}{\partial x_1 \partial x_2} & \cdots \\ \frac{\partial^2 f}{\partial x_2 \partial x_1} & \frac{\partial^2 f}{\partial x_2^2} & \cdots \\ \vdots & \vdots & \ddots \end{bmatrix}$$

The source defines the Hessian as the matrix of second-order partial derivatives and notes that under appropriate continuity conditions it is symmetric.

Example

$$f(x, y) = x^2 + 3xy + y^2$$

Then:

$$H = \begin{bmatrix} 2 & 3 \\ 3 & 2 \end{bmatrix}$$

Why do we care in ML?

The gradient tells us:

Which direction should I move?

The Hessian tells us:

How is the slope changing?

In other words:

Gradient

↓

Direction

Hessian

↓

Curvature

This becomes important for second-order optimization methods and understanding optimization landscapes.

Interview answer

The Hessian contains second-order partial derivatives and captures local curvature of a scalar function. Gradient-based methods primarily use first-order information, while second-order methods can use Hessian information.

---

10. Jacobian ⭐ ⭐ ⭐ P0/P1

This is extremely important because it generalizes the derivative to **vector-valued functions**.

Suppose:

$$f: \mathbb{R}^n \to \mathbb{R}^m$$

and:

$$f(x) = \begin{bmatrix} f_1(x) \\ f_2(x) \\ \vdots \\ f_m(x) \end{bmatrix}$$

The Jacobian is:

$$\boxed{J_f = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \frac{\partial f_1}{\partial x_2} & \cdots & \frac{\partial f_1}{\partial x_n} \\ \frac{\partial f_2}{\partial x_1} & \frac{\partial f_2}{\partial x_2} & \cdots & \frac{\partial f_2}{\partial x_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial f_m}{\partial x_1} & \frac{\partial f_m}{\partial x_2} & \cdots & \frac{\partial f_m}{\partial x_n} \end{bmatrix}}$$

Its shape is:

$$\boxed{m \times n}$$

The source explicitly defines the Jacobian this way and describes it as the generalization of the gradient for vector-valued functions.

---

11. Gradient vs Jacobian

This is a common interview question.

Suppose:

$$f: \mathbb{R}^3 \to \mathbb{R}$$

Then:

Input: 3

Output: 1

Gradient:

3 × 1

But:

$$g: \mathbb{R}^3 \to \mathbb{R}^2$$

Then:

Input: 3

Output: 2

Jacobian:

2 × 3

Memorize this:

Gradient → scalar output
Jacobian → vector output

More precisely:

$f : R^n \to R$

↓

Gradient $\in R^n$

$f : R^n \to R^m$

↓

Jacobian $\in R^{mxn}$

---

12. Jacobian Example

Suppose:

$$f(x,y) = \begin{bmatrix} x^2 + y \\ xy \end{bmatrix}$$

Then:

$$f_1 = x^2 + y$$
$$f_2 = xy$$

Therefore:

$$J_f = \begin{bmatrix} 2x & 1 \\ y & x \end{bmatrix}$$

At $(2,3)$:

$$\boxed{J_f(2,3) = \begin{bmatrix} 4 & 1 \\ 3 & 2 \end{bmatrix}}$$

---

13. Jacobian vs Hessian

Another interview favorite.

	Jacobian	**Hessian**
Function	Vector → Vector	Vector → Scalar
Derivatives	First-order	Second-order
Shape	$m \times n$	$n \times n$
Purpose	Local transformation	Local curvature

Mental shortcut:

Derivatives

|

┌──────────┴──────────┐

↓ ↓

Vector → Vector Scalar → Scalar

↓ ↓

Jacobian Gradient

↓

differentiate again

↓

Hessian

---

14. Chain Rule ⭐⭐⭐⭐⭐

This is the most important concept in this chapter for ML interviews.

The source explicitly states that the multivariable chain rule is extremely important in machine learning because neural networks are compositions of functions and the chain rule is used to calculate derivatives during gradient descent.

Simple version

If:

$$y = f\big(g(x)\big)$$

then:

$$\boxed{\frac{dy}{dx} = \frac{df}{dg}\frac{dg}{dx}}$$

---

15. Chain Rule Diagram

This is the diagram I would memorize for an interview:

x

|

| dg/dx

↓

g(x)

|

| df/dg

↓

f(g(x))

Therefore:

dy/dx

=

(df/dg)

×

(dg/dx)

---

16. Multivariable Chain Rule

For:

$$g: \mathbb{R}^n \to \mathbb{R}^m$$

and:

$$f: \mathbb{R}^m \to \mathbb{R}^l$$

we have:

$$\boxed{D(f \circ g)(a) = Df\big(g(a)\big)Dg(a)}$$

This is the matrix form given in the source.

---

17. Why Chain Rule Matters for Neural Networks ⭐ ⭐ ⭐ ⭐ ⭐

Consider a simple neural network:

Input

x

|

▼

Linear Layer

z = Wx + b

|

▼

Activation

a = σ(z)

|

▼

Loss

L(a,y)

The loss is effectively:

$$L(\sigma, (Wx + b)y)$$

We want:

$$\frac{\partial L}{\partial W}$$

The chain rule lets us propagate the derivative backward:

Forward

x —→ z —→ a —→ L

W σ

Backward

L —→ ∂L/∂a —→ ∂L/∂z —→ ∂L/∂W

That is the mathematical foundation of **backpropagation**.

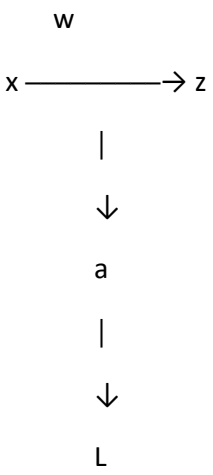
The source explicitly identifies the chain rule as the foundation of backpropagation.

---

18. Worked Backpropagation Example ⭐ ⭐ ⭐ ⭐ ⭐

Let's use an intentionally tiny network.

x = 2



Let:

$$z = wx$$
$$a = z^2$$
$$L = a$$

Suppose:

$$w = 3,\ x = 2$$

Forward pass:

$$z = 3(2) = 6$$
$$a = 6^2 = 36$$
$$L = 36$$

Now calculate:

$$\frac{dL}{dw}$$

Chain rule:

$$\boxed{\frac{dL}{dw} = \frac{dL}{da}\frac{da}{dz}\frac{dz}{dw}}$$

Calculate each:

$$\frac{dL}{da} = 1$$
$$\frac{da}{dz} = 2z = 12$$
$$\frac{dz}{dw} = x = 2$$

Therefore:

$$\frac{dL}{dw} = 1 \times 12 \times 2 = 24$$

So:

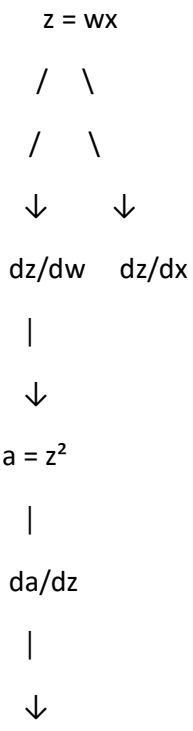
$$\boxed{\frac{dL}{dw} = 24}$$

This is the core idea behind backpropagation: **break a complicated derivative into local derivatives and multiply them together.**

---

19. Chain Rule as Computational Graph

This is even more useful for interviews:



L

Backward:

∂L/∂a

|

↓

∂L/∂z

|

┌──────┴──────┐

↓ ↓

∂L/∂w ∂L/∂x

This is essentially what automatic differentiation frameworks exploit.

---

20. Total Derivative vs Jacobian

This distinction is slightly more advanced but interview-worthy.

For:

$$f: \mathbb{R}^n \to \mathbb{R}^m$$

the total derivative is the linear map describing the best local linear approximation.

When the function is totally differentiable:

$$\boxed{Df(a) = J_f(a)}$$

The source states this relationship explicitly.

So for practical ML purposes:

Total derivative

↓

represented by

↓

Jacobian matrix

---

21. Local Linear Approximation ⭐ ⭐

This is the conceptual foundation behind all of this.

For a scalar function:

$$f(x)$$

around a:

$$f(x) \approx f(a) + f'(a)(x - a)$$

For multiple variables:

$$\boxed{f(x) \approx f(a) + \nabla f(a)^T (x - a)}$$

The source presents this as the multivariable generalization of the single-variable linear approximation.

For vector-valued functions:

$$\boxed{f(x) \approx f(a) + J_f(a)(x - a)}$$

---

22. One Example Connecting Everything

Consider:

$$f(x, y) = x^2 + xy + y^2$$

Step 1 — Partial derivatives

$$f_x = 2x + y$$
$$f_y = x + 2y$$

Step 2 — Gradient

$$\boxed{\nabla f = \begin{bmatrix} 2x + y \\ x + 2y \end{bmatrix}}$$

At $(1, 2)$:

$$\nabla f = \begin{bmatrix} 4 \\ 5 \end{bmatrix}$$

Step 3 — Directional derivative

Let:

$$v = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

Then:

$$D_v f = [4, 5] \begin{bmatrix} 1 \\ 0 \end{bmatrix} = 4$$

Step 4 — Hessian

$$H = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$$

Step 5 — Gradient descent

If we want to minimize f:

$$x_{new} = x - \eta(2x + y)$$
$$y_{new} = y - \eta(x + 2y)$$

So all these concepts connect:

f(x,y)

|

├— Partial derivatives

| ↓

| Gradient

| ↓

| Gradient descent

|

├— Second derivatives

| ↓

| Hessian

|

└── Arbitrary direction

  ↓

Directional derivative

---

23. 80/20 Priority for Your Interview

Don't spend equal time on everything.

🔴 **P0 — Must Know**

1. Partial derivative

Know:

- definition
- hold other variables constant
- simple calculation
- ML interpretation

2. Gradient

Know:

- formula
- shape
- meaning
- steepest ascent
- negative gradient → steepest descent

3. Gradient descent

Know:

$$w_{new} = w - \eta \nabla L(w)$$

4. Chain rule

Know:

$$\frac{dy}{dx} = \frac{dy}{du}\frac{du}{dx}$$

and matrix version:

$$D(f \circ g) = Df\big(g(x)\big)Dg(x)$$

5. Backpropagation

Know:

Forward pass

↓

Loss

↓

Chain rule

↓

Gradients

↓

Parameter update

These concepts give you the highest interview value.

---

24. 🟡 P1 — Know Well

- Directional derivative
- Jacobian
- Hessian
- local linear approximation
- total derivative
- gradient vs Jacobian
- Jacobian vs Hessian
- why Hessian represents curvature

---

25. 🟢 P2 — Understand Conceptually

Don't spend excessive interview-preparation time proving these unless you're interviewing for a mathematically heavy ML role:

- formal $o(\| x - a \|)$definitions
- full proof of Clairaut/Schwarz theorem
- abstract derivative operators
- rigorous proof of the multivariable chain rule
- pathological counterexamples

The source itself contains these rigorous mathematical treatments, but for an engineering-focused ML interview the practical understanding is usually more valuable. The source notes, for example, that the order of second derivatives is interchangeable under appropriate continuity assumptions.

---

26. Most Important Interview Comparisons

Partial Derivative vs Gradient

Partial derivative

↓

one variable

Gradient

↓

all variables

Gradient vs Jacobian

Scalar output

↓

Gradient

Vector output

↓

Jacobian

Gradient vs Hessian

Gradient → slope / direction

Hessian → curvature

Gradient Descent vs Gradient

Gradient

↓

information about direction

Gradient Descent

↓

algorithm using that information

Forward Pass vs Backpropagation

Forward:

input → prediction → loss

Backward:

loss → gradients → parameters

---

27. Interview Questions You Should Be Able to Answer

Q1. What is a partial derivative?

Answer:

It measures the rate of change of a function with respect to one variable while keeping the other variables fixed.

Q2. What is a gradient?

A gradient is a vector containing all partial derivatives of a scalar-valued function with respect to its input variables.

Q3. What direction does the gradient point?

The gradient points in the direction of steepest increase of the function. Therefore, the negative gradient gives the direction of steepest local decrease.

Q4. Why does gradient descent subtract the gradient?

Because the gradient points toward increasing loss. To reduce the loss, we move in the opposite direction.

Q5. What is the learning rate?

It controls the size of the parameter update.

$$w_{new} = w - \eta \nabla L$$

Q6. What happens if the learning rate is too large?

Too large

↓

overshooting

↓

oscillation / divergence

Q7. What happens if it is too small?

Too small

↓

tiny updates

↓

very slow convergence

Q8. What is a directional derivative?

It measures the rate of change of a function along an arbitrary direction.

$$D_v f = \nabla f^T v$$

Q9. What is a Jacobian?

The Jacobian is the matrix containing all first-order partial derivatives of a vector-valued function.

For:

$$f: \mathbb{R}^n \to \mathbb{R}^m$$

its shape is:

$$m \times n$$

Q10. What is a Hessian?

The Hessian is the matrix of second-order partial derivatives of a scalar-valued function. It captures local curvature.

Q11. Why is the chain rule important in deep learning?

A neural network is a composition of functions. The chain rule allows us to compute how the final loss changes with respect to parameters by multiplying local derivatives backward through the network.

This directly matches the source's explanation of neural-network layers as compositions and the role of the chain rule in calculating derivatives.

Q12. What is backpropagation?

Backpropagation is an efficient application of the chain rule that computes gradients of the loss with respect to neural-network parameters by propagating derivative information backward through the computational graph.

Q13. Is the gradient a scalar or vector?

For a scalar-valued function of $n$variables, the gradient is an n-dimensional vector.

Q14. Is the Jacobian a vector or matrix?

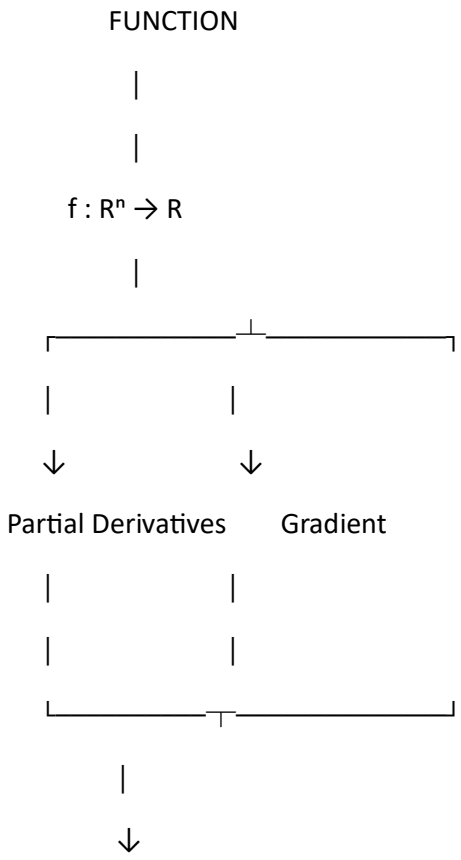
Generally, a matrix. For $f: \mathbb{R}^n \to \mathbb{R}^m$, its shape is $m \times n$.

Q15. Is the Hessian a vector or matrix?

A matrix. For a scalar-valued function with $n$inputs, it is $n \times n$.

---

28. One Diagram to Memorize Before the Interview



Directional

Derivative

|

↓

Gradient Descent

|

↓

Optimization

$f : R^n \rightarrow R^m$

|

↓

Jacobian

|

↓

Local Linear Map

Scalar $f : R^n \rightarrow R$

|

differentiate twice

↓

Hessian

|

↓

Curvature

Composition

f(g(x))

|

↓

Chain Rule

|

↓

Backpropagation

|

↓

Neural Network Training

---

29. Final 80/20 Mental Model

If the interviewer gives you a whiteboard and asks you to explain the whole topic, build your answer around this:

LOSS L(w)

|

| differentiate

↓

∇L(w) — GRADIENT

|

| opposite direction

↓

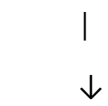
-∇L(w)

|

| step size η

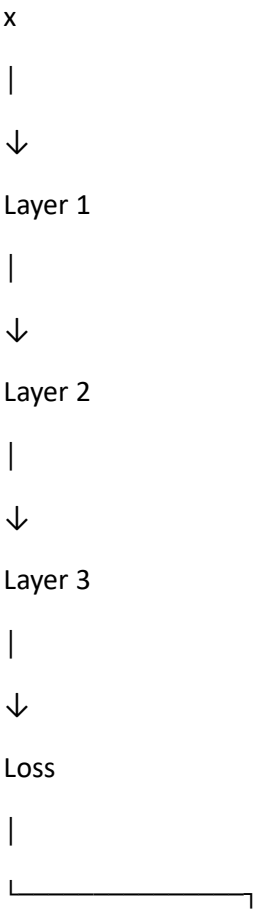
↓

$$w_{new} = w - \eta \nabla L(w)$$



GRADIENT DESCENT

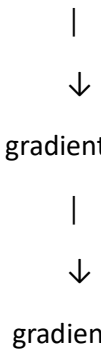
For neural networks:



CHAIN RULE



BACKPROPAGATION



The key progression from the source is essentially:

partial derivatives → gradient → local linear approximation → directional derivatives → Jacobian/Hessian → chain rule → backpropagation → gradient descent.

What I would memorize vs understand

Topic	Interview priority	What to be able to do
Partial derivative	🔴 P0	Calculate + explain
Gradient	🔴 P0	Calculate + interpret
Gradient descent	🔴 P0	Derive update + explain
Chain rule	🔴 P0	Calculate manually
Backpropagation	🔴 P0	Explain with computational graph
Jacobian	🟡 P1	Calculate + know dimensions
Hessian	🟡 P1	Calculate simple example + explain curvature
Directional derivative	🟡 P1	Calculate using gradient
Total derivative	🟡 P1	Explain local linear approximation
Rigorous proofs	🟢 P2	Understand conceptually

The highest-value exercise now is not more reading: take a tiny 2-layer neural network and manually calculate **forward pass → loss → partial derivatives → chain rule → backpropagation → gradient update**. That single exercise ties most of this chapter together.